Does Architecture Help or Hinder AI Coding Agents?

A Preregistered Experiment with Layered and Hexagonal Variants of an Evolving Java Service

Tomasz Pisarewski
Fide-Soft
tomasz.pisarewski@fide-soft.com

Draft of November 7, 2026

Abstract. Software architecture exists to help people understand and change code. Coding agents now make a growing share of those changes. Two questions follow. Does a structure designed for human cognition help or hinder an agent? And does the agent keep that structure intact as it works? We report a preregistered controlled experiment (OSF `r5t3d`). We built an HL7 FHIR R4 REST service in Java 21 and Spring Boot 3.5 in three behaviorally equivalent variants: no explicit architecture (None), layered (Layered), and hexagonal, i.e. ports and adapters (Hexagonal). All three pass the same 79-test black-box suite. Two open-weights models, Laguna S 2.1 and Qwen3-Coder-30B-A3B, run in the mini-swe-agent scaffold. Each implements the same 24 product tickets in a fixed order, every ticket on top of its own previous changes. A preregistered cost rule dropped the fifth run, leaving four runs per cell (24 trajectories, 576 attempts, none missing). Hidden acceptance tests measure success. Tokens measure cost. After every task we measure erosion with an architecture-agnostic index and with ArchUnit conformance. Architecture changed neither task success nor token cost. Success (H3) did not differ (hexagonal minus none: −1.0 percentage points, 95% CI [−6.8, 4.7], Holm-adjusted $p = 1.00$), nor did total tokens (H1). The capable model solved 56%, 57% and 55% of tasks in None, Layered and Hexagonal, although in Hexagonal it modified about twice as many files per task. Architecture did change what happened to the code. Contrary to H2, neither architecture eroded more slowly than None. The layered code base eroded fastest, and the hexagonal one eroded significantly more slowly than the layered one (Holm-adjusted $p = 0.031$). Conformance violations tended to accumulate (H6, not significant), with no difference between Layered and Hexagonal. In exploratory analyses, three of four hexagonal trajectories of the capable model never violated their own rules, but hexagonal attempts more often left a project that did not build (30% vs. 11% in None). The weaker model failed at the floor (2–5% success), so the capability interaction (H4) cannot be interpreted. Post-hoc checks, not preregistered, showed that all 24 tasks are solvable in every variant, that re-grading all 403 failures changed no outcome, and that the capable model's successes are genuine implementations. The whole experiment ran on a laptop with Docker and cost USD 24.25 in API fees. We release all variants, tasks, hidden tests, harness, transcripts and per-task snapshots.

Keywords: coding agents; software architecture; architecture erosion; hexagonal architecture; controlled experiment; preregistration; open science.

1 Introduction

Software architecture is a tool for human cognition. Modules hide design decisions so that a person can change one part of a system without understanding the rest [24]. Layers and ports separate concerns so that the developer knows where a change belongs and what it may touch [3, 7, 10]. These structures cost something: more files, more indirection, more mapping code. We pay that cost because it buys comprehension and controlled change for human maintainers.

LLM-based coding agents now take on a growing share of maintenance and feature work. They read code through a shell, not through an IDE and a mental model built over months. Two things are unclear. First, does the indirection that helps humans help agents, or does it tax them? Existing evidence comes from single tasks on small code bases (§2) and covers neither realistic project size, nor Java, nor sustained evolution. Second, do agents preserve an architecture while they evolve the code? Architecture erosion, the growing gap between intended and implemented architecture, is a classic problem of long-lived systems [8, 17, 19, 25]. Agents may speed it up. They optimize for local correctness, the ticket at hand and its tests, and may drift from global architectural intent [5]. A recent benchmark has agents extend their own code over checkpoints and measures erosion at each step [23]. None of these studies vary the architecture while holding behavior constant.

We run that experiment. One FHIR R4 service exists in three behaviorally equivalent variants (NONE, LAYERED, HEXAGONAL) that pass the same black-box suite. An agent implements 24 product tickets in order, each on top of its own previous result, and we measure success, cost and erosion after every task. The design, hypotheses, analysis code and sample size were frozen and registered before any confirmatory run. We ask three research questions:

RQ1 *Cost.* Does architecture change how much work an agent needs per task? (H1; by task type, H5)

RQ2 *Success.* Does architecture change whether the agent completes the task, and does that depend on model capability? (H3; H4)

RQ3 *Erosion.* Does an explicit architecture slow structural decay under agent-driven evolution, and do agents keep to the architecture's own rules? (H2; H6)

In short, for a capable agent we found no effect of architecture on success or token cost, but a clear difference in what happened to the code over 24 tasks: the layered code base eroded fastest, and the hexagonal one held its structure best. The weaker model failed at the floor, so the capability question (H4) remains open.

Contributions.

1. **Evidence.** A preregistered test of architecture effects on agents under sustained evolution. With a capable open-weights model, success and tokens did not differ between architectures, while the erosion rate did (HEXAGONAL slower than LAYERED); several preregistered predictions were not supported, and we report them as such (§4).
2. **Controlled, equivalent variants.** Three architectures of one non-trivial service (2.2–4.1 kLOC), with an identical external contract, schema and dependency set, admitted only after a mechanical equivalence gate. Building them in parallel exposed 13 specification ambiguities, which became binding clarifications.
3. **Cumulative evolution with erosion tracking.** Agents work on their own earlier output across 24 tickets. Every task is a git snapshot, graded against all hidden tests so far and measured with a pinned erosion pipeline (ArchUnit, CK, PMD/CPD, SpotBugs).
4. **Preregistration.** Hypotheses, tests, sample size and the analysis script were frozen (git tag `prereg-v1`) and registered on OSF before data collection. The design and methods sections of this paper were drafted before the results were known. Four deviations and a set of post-hoc validity checks are logged and reported.
5. **Open, reproducible, cheap.** Open-weights models, an offline Docker sandbox and an unmodified public scaffold (mini-swe-agent 2.4.6). The full experiment cost USD 24.25, about USD 0.04 per attempt. All materials, transcripts and snapshots are released.
6. **Lessons for low-budget agent experiments** (§7): failure modes of free and paid endpoints, scaffolds, graders and capability checks that we hit before and during data collection.

2 Related Work

Coding-agent benchmarks. SWE-bench [15] established repository-level issue resolution with hidden FAIL_TO_PASS and PASS_TO_PASS tests. We follow that grading scheme. SWE-agent [40] showed that the agent–computer interface matters. mini-swe-agent [33] reduces the scaffold to a bash-only loop and is the reference scaffold of the SWE-bench Bash Only leaderboard [34]. We use it so that scaffold effects stay small and identical across models. Java issue-resolution benchmarks exist [41, 42]. Like most benchmarks they evaluate independent tasks on fixed snapshots of real repositories, so they cannot separate the effect of architecture from everything else that differs between repositories.

Long-horizon evolution and degradation. SWE-EVO [16] poses long-horizon evolution tasks derived from release notes. SlopCodeBench [23] is the closest design to ours: agents extend their own code over checkpoints, and erosion and verbosity are measured at every step. It does not manipulate architecture while holding behavior constant, and that manipulation is what this study adds. The notions of *agentic entropy*, introduced in a position paper [5], where agents optimize local correctness and drift from global architectural intent, and *coherence debt* in repository-scale tasks [21] give conceptual framings for H2 and H6.

Code properties that affect agents. A growing line of work holds the agent fixed and varies the code. Trivedi and Schmitt [36] build minimal pairs that differ in code cleanliness and measure success and token cost with hidden tests. They leave open whether such effects compound over long-horizon evolution, which our cumulative design addresses for architecture. Borg et al. [2] relate file-level CodeHealth to LLM refactoring outcomes. Ustynov [37]

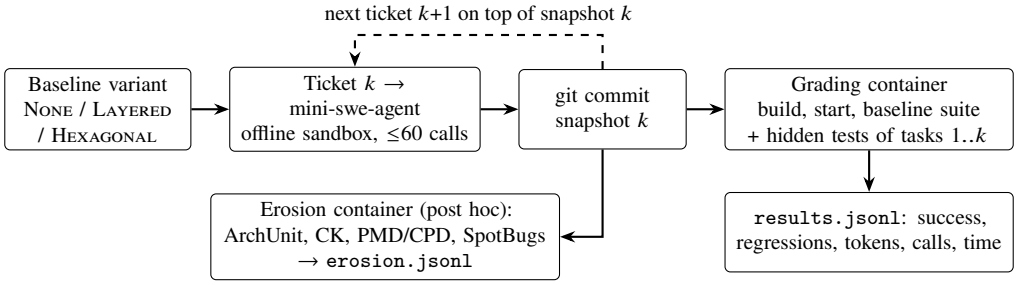


Figure 1: One trajectory. The agent never sees the hidden tests. Grading and erosion measurement run on the committed snapshot in separate offline containers.

revisits conventions written for human readers in light of agentic development and leaves architecture to future work. *Theory of Code Space* [29] asks whether agents can build an architectural map of a code base. All of these work with single tasks or single snapshots.

Architecture and agents. Three recent works are closest in topic. *Architecture as Capability Equalizer* [4] varies how an architecture is *specified* to six models, including ArchUnit-style rules, and finds that structured specifications help weaker models most. It does not vary the architecture of the code. A pilot study [30] has three models build a hexagonal microservice once and reports that, without automated architectural linting, the smallest model accumulates structural debt fastest. It studies one style with no controlled alternative and no sustained evolution. In grey literature, Stoyanov [32] compares a flat and a hexagonal version of one Java system on 9 + 6 tasks and reports that the hexagonal version took 37.8% longer to reach acceptance on the nine feature tasks (about 8% longer on the six challenges). That comparison involves one system and one author and was not preregistered. Together these works motivate H1, H2 and H6 but do not test them under control.

Architecture erosion. Perry and Wolf [25] distinguish *erosion*, which violates the architecture, from *drift*, which is insensitivity to it. De Silva and Balasubramaniam [8] survey methods to control erosion, including conformance checking against rules, which we implement with ArchUnit [35]. Li et al. map the erosion literature [19] and report practitioners' views of its causes and symptoms [18]. Common symptoms (dependency cycles, rising coupling, duplication, rule violations) are what our erosion index aggregates. We measure them with established metrics: CK [1, 6], Martin's distance from the main sequence [20], PMD/CPD [26] and SpotBugs [31]. To our knowledge erosion research has so far studied human developers. Agents as the source of erosion are new.

3 Study Design

This section summarizes the frozen protocol (PREREGISTRATION.md at tag `prereg-v1`). Phase 1, reported here, covers the FHIR domain. A second domain (order management) is built with the same method and will be registered separately as Phase 2, where domain generality (H7) is tested. Figure 1 shows the data flow for one trajectory.

3.1 Application and variants

The application is a FHIR R4 REST facade [12] over its own persistence. It covers five resource types (`Patient`, `Practitioner`, `Encounter`, `Observation`, `Appointment`) with read, create, update and a defined subset of search parameters, resource versioning (`meta.versionId`, `ETag`) and `OperationOutcome` errors. Every variant uses Java 21, Spring Boot 3.5.16, Spring Data JPA, H2, Flyway and the HAPI FHIR *structures and parser only* (`hapi-fhir-structures-r4` 8.12.1). HAPI server modules are excluded, because they impose their own architecture and would remove the manipulated variable. Seed data is synthetic (Synthea [38]).

We chose Spring Boot 3.5 rather than 4.x on purpose. Boot 4 (November 2025) changed core APIs (Jackson 3, modularized starters) and is newer than much of the models' training data. It would add a framework-novelty confound unrelated to architecture.

The variants differ only in code organization (ARCHITECTURES.md):

- NONE: idiomatic "tutorial-style" Spring. Controllers parse, validate, apply rules, map FHIR resources and call Spring Data repositories directly. Packages `controller`, `entity`, `repository`, `util`. Realistic, not deliberately bad.
- LAYERED: package-by-layer `web` → `service` → `repository` (+ `entity`). HAPI types are confined to `web`.

Table 1: Size profile of the three baselines at the equivalence gate (2026-10-07). All pass the same 79-test acceptance suite.

Variant	Main LOC	`.java` files	Packages	Own tests (incl. ArchUnit)	`AGENTS.md` words
NONE	2243	27	4	8	173
LAYERED	2677	55	5	14	183
HEXAGONAL	4064	104	12	16	185

Table 2: Task strata. Units = architectural units a change is expected to touch.

Stratum	Tasks	Example	Units
D domain rule	T01 T05 T09 T13 T17 T21	T01: body-weight Observations must use UCUM `kg` or `g`, else 422 `business-rule`	1
F feature	T02 T06 T10 T14 T18 T22	T02: `Observation?date=` search with prefixes; listed in the CapabilityStatement	3+
C cross-cutting	T03 T07 T11 T15 T19 T23	T03: optimistic locking with `If-Match` on every `PUT`; 412 on conflict	many
I integration	T04 T08 T12 T16 T20 T24	T04: POST critical glucose Observations to a configured URL; failure must not fail the create	new adapter

- HEXAGONAL: ports and adapters after Hombergs [14] (reference implementation *buckpal*). A pure-Java `domain` with its own clinical model, `application` with inbound/outbound ports and use-case services, and `adapter.in.web`, `adapter.out.persistence` and `config`. FHIR mapping lives only in the web adapter.

The FHIR domain makes the placement of the standard model itself an architectural decision. In NONE HAPI resource classes run through the whole code base. In LAYERED they stop at the service boundary. In HEXAGONAL the domain has its own model and the FHIR mapping sits in one adapter.

Equivalence gate. A variant was admitted only if (i) it passes the shared black-box acceptance suite (79 tests over HTTP against the running jar, with FHIR R4 validation of every response), (ii) its Flyway schema is byte-identical and its dependency set exactly matches the other variants (ArchUnit in test scope is the only extra), (iii) it has zero violations of its own ArchUnit rules, and (iv) its size profile is published (Table 1). The size differences are intrinsic to the styles (ports, mapping, a separate domain model). We report them and do not equalize them. Each variant also contains an `AGENTS.md` with the same build/test commands and a neutral, package-level layout description of equal length (±10%) and no task hints.

Construction. The specification (`fhir/SPEC.md`) is the single source of truth for behavior. Three AI agents built the three variants in parallel from it, and the author reviewed them. Whenever two builders interpreted the specification differently, the ambiguity became a binding clarification with its own acceptance test, and all variants were re-gated. This produced 13 clarifications (C1–C13), for example the order of checks on `PUT` (parse error before id mismatch before unknown id before business rules), the status for unsupported HTTP methods on known types (405), and the handling of codes that are valid in FHIR R4 but outside the supported subset. The gate caught four real behavioral divergences (C1, C7, C9, C11). We see this as a side result: parallel independent implementation is an effective way to find specification gaps.

3.2 Tasks

There are 24 tasks, written as product tickets. They describe externally observable behavior and never name files, classes, packages or layers, so all three variants receive identical text (1861 words in total). HTTP contracts and configuration property names are part of the external interface and may be named. Every ticket ends with acceptance criteria that the agent can see. The tasks fall into four strata of six, interleaved in a fixed order that mimics product evolution (Table 2).

Hidden tests. Each task has hidden black-box acceptance tests (141 test methods in total, 3–8 per task), the same file for all variants. They run over HTTP against the running application. FAIL_TO_PASS tests fail on every baseline. Guard tests (PASS_TO_PASS) pass on the baseline and stop over-reaching solutions. For integration tasks the grader starts the application with the relevant property pointing to a mock server controlled by the test. `harness/validate_tasks.py` checks on each baseline that a task's FAIL_TO_PASS tests fail, its guard tests pass and the baseline suite passes. Before registration this check was logged for 12 tasks (`data/validate_phase1.log`). The preregistration also planned reference solutions, but only a partial one (12 tasks, NONE) existed

at registration. The full validation of all 24 tasks and reference solutions for all three variants were completed after data collection (§4.9).

Dependencies and superseded tests. Tasks were designed to be independent: 22 have no dependency, and two have a soft one that their hidden tests do not need (T19 on T03, T20 on T06). Some tickets legitimately change baseline behavior. For example, T18 makes `_count` a valid search parameter, and several tasks extend the CapabilityStatement that a baseline test checks exactly. Nine of the 24 tasks therefore declare which baseline tests they *supersede*. From that task on, those tests are no longer counted as regressions (§3.4).

3.3 Agent and models

Scaffold. mini-swe-agent 2.4.6 [33] with its `DefaultAgent` and a single bash tool through native tool calling. The prompt is identical in every condition and neutral about architecture. It asks the agent to read `AGENTS.md`, find the relevant code, make the change, build and test, and submit. The agent may run the visible tests (`mvn test`) and start the application, and it never sees the hidden tests. The budget is 60 model calls per task. *Success* means solved within that budget: the hidden tests pass on the committed state, whatever the stop reason. Each shell command times out after 600 s. There is no wall-clock limit, because provider latency must never become an agent failure.

Models. Both arms use open-weights models, served through OpenRouter at temperature 0.2 with at most 16,384 output tokens per call:

- *strong*: Poolside Laguna S 2.1 [27] (118B total / 8B active coding mixture-of-experts, OpenMDW-1.1 weights), run *without* extended thinking. With thinking on, the serving provider ignored both the reasoning budget and `effort: low`. Calls ran to the output cap without a tool call, and in a shakedown 6 of 17 attempts ended in repeated format errors. With thinking off there were no format errors in 120 calls. This decision was made on reliability alone, before registration.
- *weak*: Qwen3-Coder-30B-A3B-Instruct [28]. A preregistered floor rule would have replaced it with Qwen3-Coder-Next if it had solved fewer than 30% of pilot attempts in None. The rule was not triggered. Under the cumulative protocol the model nevertheless failed at the floor (§4.5).

Open weights matter for reproducibility. Hosted endpoints change or disappear (§7), but anyone can re-run the study with the same weights. The serving provider is logged per call. In the pilot, all strong-arm calls were served by Poolside and all weak-arm calls by SiliconFlow.

3.4 Harness

Sandbox. Each attempt runs in a fresh container of a pre-warmed image (`archagents/sandbox:1`, Temurin JDK 21, Maven with every dependency cached) with `--network none`, 2 CPUs and 3 GB RAM. Maven runs offline. The trajectory's repository is mounted as the workspace. Every command is wrapped in `timeout -k 5 590` inside the container, so a hung build or a forgotten server is killed before the scaffold gives up on the command.

Grading. After the agent stops, the harness commits the workspace. The grader exports the committed tree (`git archive`), builds it in a separate offline container, starts the jar, and runs the baseline acceptance suite plus the hidden tests of the current and all earlier tasks. App configuration accumulates, so earlier integration tasks keep being checked against their mock servers. A task is a *success* if the snapshot builds, the application starts, and every hidden test of the current task passes. A test is a *regression* if it passed at the previous grading and fails now. Tests of earlier tasks that the agent never solved are recorded as *still failing*, not as regressions. Superseded baseline tests are excluded from both.

Infrastructure failures. Provider errors, container crashes and harness exceptions are not agent failures. Only four exit states count as agent outcomes: submitted, step limit reached, time exceeded, and repeated format errors. Any other exit rolls the workspace back to the last commit, and the attempt is retried later, up to three times. After that it is recorded as missing. A trajectory with more than 20% infrastructure failures is re-run from scratch. No data are excluded because of their outcome values.

Scheduling. Task order is fixed. Execution is interleaved: for each task position, the harness visits the trajectories in a seeded random order. Provider drift, throttling or a withdrawn model therefore hits all architecture arms equally. As a logged deviation (§3.7), Phase 1 ran one such interleaving worker per run, two to three in parallel. Each worker contained all six architecture × model trajectories of its run. After a host interruption, the rest of run R4 ran in two workers, one per model, each interleaving the three architectures.

Table 3: Components of the erosion index (PREREGISTRATION §4.3).

m	Component	Source
1	package-level dependency cycles in `org.archagents.fhir.*`	ArchUnit slices
2, 3	mean and max coupling between objects (CBO)	CK
4	mean lack of cohesion (LCOM)	CK
5	mean normalized distance from the main sequence $D = \|A + I - 1\|$	ArchUnit metrics
6	static-analysis warnings (PMD violations + SpotBugs bugs)	PMD, SpotBugs
7	duplicated lines (clones ≥100 tokens)	CPD

3.5 Measures

Success and cost (per attempt). Success (binary), regression flags, stop reason, total tokens (prompt + completion, including cached prompt tokens), model calls, wall time, files changed, and added/deleted lines. Total tokens is the primary cost measure. We also report a recomputed USD cost that prices cached tokens separately.[1]

Erosion (per committed snapshot). `harness/erosion.py` measures every snapshot post hoc in a second pinned image (`archagents/sandbox:2`). The image adds ArchUnit 1.5.1, CK 0.7.0, PMD/CPD 7.28.0 (`quickstart` rule set) and SpotBugs 4.10.4 (default effort, medium threshold), all checked against published SHA-256 digests. Because a snapshot is deterministic, post-hoc measurement gives the same values as measuring inside the loop. CK 0.7.0 cannot parse Java records. The tool therefore parses at JLS 21 and runs CK on a copy in which each record is rewritten as a final class with the same fields. With this change, CK reports no errors on any baseline. The index has seven components (Table 3).

For trajectory t, task index k (0 = baseline) and component m, let $x_{t,k,m}$ be the metric value on the committed snapshot. Then

$$d_{t,k,m} = x_{t,k,m} - x_{t,0,m}, \qquad s_m = \mathrm{SD}_t\left(d_{t,K_t,m}\right), \qquad E_{t,k} = \frac{1}{|M^*|} \sum_{m \in M^*} \frac{d_{t,k,m}}{s_m}, \tag{1}$$

where K_t is the last compiling snapshot of t, the standard deviation pools all trajectories across arms and models, and $M^* = \{m : s_m > 0\}$. A component with zero pooled spread is dropped and reported. There is no centering, so $E = 0$ still means "no change since baseline" (centering would add a constant per component and not change any slope). The H2 outcome is the OLS slope β_t of $E_{t,k}$ over k. Snapshots that do not compile have no metric values. They are left out of the slope fits, and their number per arm is reported.

Conformance (H6). For Layered and Hexagonal, the number of violations of the variant's own layer or hexagon rules (ARCHITECTURES.md). They are evaluated by a frozen copy of the baseline rules in the measurement tool, never by the copy in the snapshot, which the agent may have edited. The general checks (field injection, HAPI server modules) and the cycle rule are not counted, since cycles enter the erosion index. A rule whose target package was deleted counts as one violation. The H6 outcome is the OLS slope of this count over k.

3.6 Hypotheses and analysis

Table 4 lists the hypotheses and their tests. All primary tests are permutation tests on the unit of inference [11], implemented in `analysis/analyze.py`, which was frozen with the registration and dry-run on the pilot data. Everything is seeded (20261012).

Task-level tests (H1, H3, H4, H5). The task is the unit of inference. For each task and arm we pool all attempts over runs and models and compute the success rate (H3) or the mean log(total tokens + 1) (H1). For each contrast, the 24 per-task differences enter a two-sided sign-flip permutation test on their mean, with 100,000 random sign patterns including the observed one. When there are 16 or fewer tasks, as in the per-stratum H5 contrasts, all patterns are enumerated. H4 applies the same test to per-task interaction contrasts (weak-model effect minus strong-model effect, in percentage points). H5 permutes stratum labels to test whether the Hexagonal−None cost difference is larger in strata F, C and I than in D. Secondary H1 analyses repeat the test on successful attempts only, model calls, files changed, wall time and recomputed USD.

[1] Prices per million tokens at registration (input / output / cache read): strong 0.09 / 0.18 / 0.009, weak 0.07 / 0.28 / 0.07 USD.

Table 4: Preregistered hypotheses for Phase 1 and their primary tests. Contrasts: LAYERED−NONE, HEXAGONAL−NONE, HEXAGONAL−LAYERED, Holm-corrected within each hypothesis. H7 (domain generality) needs Phase 2.

	Prediction	Sides	Primary test
H1	cost: NONE < LAYERED < HEXAGONAL (total tokens)	directional	sign-flip test on per-task differences of mean log tokens (24 tasks)
H2	erosion slope lower in LAYERED and HEXAGONAL than in NONE	directional	label permutation of per-trajectory index slopes, stratified by model
H3	success rate differs between architectures	two-sided	sign-flip test on per-task success-rate differences
H4	HEXAGONAL−NONE success gap larger for the weak model	directional	sign-flip test on per-task interaction contrasts
H5	HEXAGONAL cost penalty largest for multi-unit strata (F, C, I), smallest for D	directional	per-stratum H1 contrasts; permutation of the (F,C,I) vs. D contrast
H6	conformance violations accumulate (slope > 0); LAYERED vs. HEXAGONAL two-sided	mixed	as H2 on violation-count slopes

Trajectory-level tests (H2, H6). We fit one OLS slope per trajectory and compare arms with a two-sided permutation test that shuffles architecture labels within model strata (10,000 relabelings; 10 trajectories per arm as designed, 8 after the cost rule dropped run R5). For the H6 claim that violations accumulate, a sign-flip test on the per-trajectory slopes tests whether their mean differs from zero.

Inference criteria. $\alpha = 0.05$. The three architecture contrasts within each hypothesis are Holm-corrected [13]. A directional hypothesis counts as supported only if the corrected test is significant *and* the effect has the predicted sign. All effects come with 95% percentile CIs from a cluster bootstrap over tasks with 10,000 resamples [9]. Secondary mixed models estimate adjusted effects: a logistic GLMM `success ~ architecture * model + (1|task) + (1|trajectory)`, LMMs with the same structure for log tokens, and `erosion_index ~ architecture * task_index + (task_index|trajectory)`. A sensitivity analysis adds `dependency_failed` to the success model. Where models and primary tests disagree, both are reported. The frozen script lacked these models. They were added before any Phase 1 outcome was inspected (DEVIATIONS.md, 2026-10-13), with two choices the registration did not fix: the logistic GLMM is fitted by variational Bayes (`statsmodels BinomialBayesMixedGLM`) and reported as posterior means and standard deviations of the log-odds, and the LMMs are fitted by REML (`statsmodels MixedLM`).

Sample size and sensitivity. The design is 3 architectures × 2 models × 5 runs = 30 trajectories of 24 tasks, i.e. 720 attempts. The size was set by resources: the study is self-funded with an API budget of about USD 30. A simulation (`analysis/power_sim.py`; logistic model with task and trajectory random effects calibrated to the pilot) gave the primary H3 test with 24 tasks × 5 runs a power of 0.93 at $\alpha = 0.05$ for a 15 percentage-point difference, and 0.86 at the Holm-adjusted $\alpha = 0.0167$. For 10 points the power is 0.67 and 0.53. Smaller differences are reported as inconclusive, not absent. The simulation also showed that adding tasks raises power more than adding runs, because the task is the unit of inference. This is why we chose 24 tasks × 5 runs over 12 × 10 at equal cost. A preregistered, outcome-blind cost rule would drop run R5 if the forecast spend after R1 exceeded USD 30. *Outcome:* the rule ran when R1 completed (2026-10-14, 05:20 UTC). After 438 attempts the mean cost was USD 0.042 per attempt, a forecast of USD 30.24 for 720 attempts, so R5 was dropped as preregistered. The realized design is 3 × 2 × 4 = 24 trajectories, 576 attempts. Re-running the registered simulation with 4 runs (3,000 simulated data sets per effect size; not preregistered) gives a power of 0.88 at $\alpha = 0.05$ and 0.77 at $\alpha = 0.0167$ for 15 points, and 0.61 and 0.46 for 10 points. These figures assume, as the registered simulation did, a mean success near 75% and two informative models; §6 explains why the realized sensitivity is lower.

3.7 Preregistration and open science

Preregistration fixes the confirmatory analysis before the data exist and so separates it from exploration [22]. The open-ended idea was registered on OSF on 2026-10-05 (`https://osf.io/w467e`). The full protocol was registered on 2026-10-13 at 09:25 UTC with the OSF Preregistration template (`https://osf.io/r5t3d`); it stayed under embargo until data collection ended. The registered protocol motivates the directional hypotheses in part with an earlier exploratory study of ours on small Python repositories. We later found methodological problems in that study, so this paper does not use its results as evidence. The registered materials are git tag `prereg-v1`, commit `3d347cb`. Before registration only harness-debugging data existed: three pilot tasks P1–P3 in all six cells

Table 5: Data collection summary (top) and task success rate by architecture and model, pooled over runs and tasks (bottom; successes / attempts in parentheses).

Runs completed (cost rule)	4 of 5 (R5 dropped)
Trajectories / attempts	24 / 576
Missing after 3 infra retries	0
Infrastructure retries	0 (1 rerun after host stop)
Trajectories re-run (>20% infra)	0
Submitted / step limit / format err.	216 / 345 / 15
Graded snapshot did not build, N/L/H: strong	11 / 15 / 29 (of 96)
weak	86 / 88 / 89 (of 96)
Main code does not compile (erosion), N/L/H	96 / 96 / 111 (of 192)
Providers (strong / weak)	Poolside / SiliconFlow
Total cost; per attempt	USD 24.25; USD 0.042

	None	Layered	Hexagonal
strong	56.2% (54/96)	57.3% (55/96)	55.2% (53/96)
weak	5.2% (5/96)	2.1% (2/96)	4.2% (4/96)
pooled	30.7% (59/192)	29.7% (57/192)	29.7% (57/192)

(18 attempts) and one shakedown trajectory over T01–T24 (None, strong model). Both are excluded from every analysis. The models, settings, sandbox images, tasks, hidden tests and the primary tests did not change after the freeze. Four deviations from the protocol are logged in DEVIATIONS.md. None changes a primary test. The post-hoc validity checks run after data collection are logged there as well and reported in §4.9.

1. *Parallel workers* (2026-10-13). Execution ran in two to three parallel workers, one per run, instead of a single queue over all 30 trajectories, because a sequential queue would take about 70 hours. Every worker interleaves all arms, so the protection against drift is kept.
2. *Completed secondary analyses* (2026-10-13). The GLMM/LMMs, the `dependency_failed` sensitivity analysis and the one-sample H6 test, all specified in the registration but missing from the frozen script, were added to `analysis/analyze.py` while data collection was running and before any outcome was inspected.
3. *Host interruption* (2026-10-14). All collection processes stopped at about 07:25 UTC when the host session ended, with R1–R3 complete and R4 at 30 of 144 attempts. The cost rule had already dropped R5. R4 resumed at 12:50 UTC in two workers, one per model, each still interleaving the three architectures. The interrupted attempt was rolled back and rerun, and the orchestrator now treats the cost-rule decision as final across restarts.
4. *Erosion snapshot source* (2026-10-14). In one trajectory (Layered, weak model, R4) the agent ran `git reset --hard` during T21 and removed the T20 commit from the branch history. `harness/erosion.py` now measures the commit that was graded (recorded in `results.jsonl`) instead of walking the history. This restores one missing measurement (599 → 600 snapshots) and leaves all others unchanged.

4 Results

All numbers come from `data/phase1/analysis.json` (primary tests and mixed models, produced by the frozen `analysis/analyze.py`) or directly from `results.jsonl` and `erosion.jsonl`. Every preregistered test is reported, whether significant or not. Analyses that were not preregistered as confirmatory are marked *exploratory*; their p-values are not corrected and only describe this data set.

4.1 Data collection

Data were collected from 2026-10-13, 09:30 UTC, five minutes after registration, to 2026-10-14, 19:19 UTC, with one interruption of about 5.4 hours when the host session ended (Table 5; deviation 3 in §3.7). No attempt failed on infrastructure, so no attempt is missing and no trajectory was re-run. The one attempt cut off by the host stop was rolled back and rerun. At the level of single model calls, the API wrapper retried 7 transient provider errors (HTTP 500/520, timeouts returned with HTTP 200, one connection reset); none ended an attempt. Every strong-model call was served by Poolside and every weak-model call by SiliconFlow. Of all prompt tokens, 91.7% were served from the provider cache.

The two models differed sharply. The strong model solved 56% of its attempts. The weak model solved 11 of 288 (3.8%), all of them on T01, T02, T07 or T09. In all 12 weak-model trajectories the snapshot stopped building at T01, T02 or T03, and only 8 of the 259 later weak-model attempts produced a building snapshot again. Because each task builds on the previous one, a broken build is close to an absorbing state: every later hidden test fails. The

Table 6: Task-level primary tests (24 tasks): mean per-task difference, 95% cluster bootstrap CI, sign-flip p, Holm-adjusted p. H3 in percentage points of success; H1 in log total tokens (positive = first arm more expensive); H4 is the interaction contrast weak-model effect minus strong-model effect in pp (no Holm correction; prediction: positive gap for the weak model, i.e. negative for HEXAGONAL−NONE). H4 is not interpretable because the weak model is at the floor (§4.5).

Contrast	Estimate	95% CI	p	p_{Holm}
H3 success (pp), two-sided				
LAYERED−NONE	−1.04	[−6.25, 4.17]	0.850	1.000
HEXAGONAL−NONE	−1.04	[−6.77, 4.69]	0.866	1.000
HEXAGONAL−LAYERED	0.00	[−5.73, 6.25]	1.000	1.000
H1 cost ($\Delta \log$ *tokens, all attempts), directional*				
LAYERED−NONE	0.003	[−0.081, 0.090]	0.948	1.000
HEXAGONAL−NONE	−0.014	[−0.110, 0.089]	0.798	1.000
HEXAGONAL−LAYERED	−0.017	[−0.078, 0.043]	0.603	1.000
H4 capability interaction (pp), directional				
LAYERED−NONE	−4.17	[−14.58, 6.25]	0.566	n/a
HEXAGONAL−NONE	0.00	[−12.50, 12.50]	1.000	n/a
HEXAGONAL−LAYERED	4.17	[−8.33, 15.63]	0.628	n/a

strong model also broke the build at times (11–30% of attempts, Table 5) but usually recovered within a few tasks. Of the API cost, USD 6.66 was spent by the strong arm and USD 17.59 by the weak arm.

4.2 H3: Success

Table 5 gives success rates per cell and Table 6 the primary test. H3 is *not supported*: no contrast comes close to significance, and all three point estimates lie within about one percentage point of zero. The confidence intervals exclude pooled differences larger than about 7 pp in either direction. Because the weak arm contributes almost no successes, a pooled difference corresponds to roughly twice that difference within the strong model. Restricted to the strong model (exploratory), HEXAGONAL−NONE is −1.0 pp with a 95% CI of [−12.5, 10.4]. Differences of 10–12 pp for a capable agent are therefore not ruled out. The secondary GLMM gives a HEXAGONAL vs. NONE odds ratio of 1.07 for the strong model (reference level; approximate 95% posterior interval 0.66–1.71) and 1.17 for LAYERED (0.69–1.97). The preregistered `dependency_failed` sensitivity analysis is degenerate: no task declares a hard dependency (two have soft dependencies their hidden tests do not need, §3.2), so the covariate is zero for every attempt and the model cannot be estimated. Figure 2a shows success per architecture and model.

4.3 H1: Cost

H1 is *not supported*. Total tokens per attempt did not differ between architectures (Table 6, Figure 2b). The largest estimate, HEXAGONAL−NONE, is −0.014 in log tokens, about 1.4% *fewer* tokens, with a CI from about 10% fewer to 9% more. The LMM estimate for HEXAGONAL vs. NONE is 0.004 (95% CI $[-0.112, 0.121]$, $p = 0.94$), but its optimizer reported convergence problems. The share of attempts that hit the 60-call budget was 55.7% / 60.4% / 63.5% (NONE / LAYERED / HEXAGONAL; strong model 57.3% / 64.6% / 65.6%, weak model 54.2% / 56.2% / 61.5%). Most attempts are therefore censored at the budget, and total tokens largely measure how much context the agent carried per call (§6).

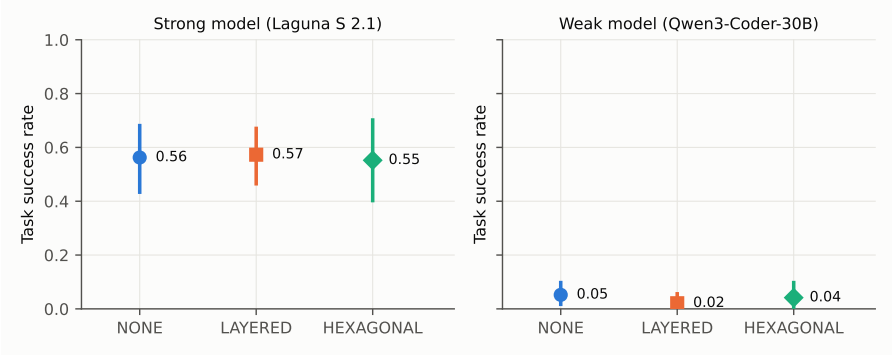
Among the secondary measures (Table 7), only two differ. The agent modified more files in the structured variants: 1.9 more per attempt in LAYERED and 3.0 more in HEXAGONAL than in NONE (both Holm $p < 0.001$). For the strong model the means are 4.8, 6.7 and 9.5 files. Wall time was *shorter* in HEXAGONAL, by 59 s per attempt vs. NONE and 66 s vs. LAYERED. Wall time also reflects provider latency and build times inside the sandbox, so we do not interpret it further. Model calls, tokens of successful attempts and recomputed USD did not differ.

4.4 H5: Cost by task stratum

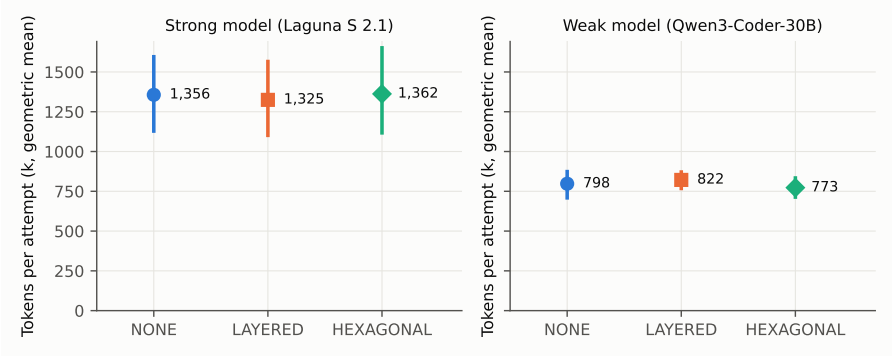
H5 is *not supported* (Table 7). The HEXAGONAL−NONE cost difference was not larger in the multi-unit strata. The point estimate even has the opposite sign: HEXAGONAL was relatively cheaper in strata F, C and I than in D (difference −0.19 in log tokens, permutation $p = 0.117$). No single stratum shows a significant difference. Figure 3 shows success by stratum for comparison: domain-rule tasks (D) were solved most often in every variant, and within each stratum the variants differ by at most 6 percentage points (pooled over both models).

4.5 H4: Capability interaction

H4 is *not interpretable*. The test returns no evidence of an interaction (Table 6; HEXAGONAL−NONE interaction 0.0 pp, $p = 1.00$), but the weak model solved only 2–5% of tasks in every arm, with 90–93% of its graded snapshots failing to build. An architecture effect on the weak model cannot show up when there is almost no



(a) task success rate, mean over tasks with 95% bootstrap CI over tasks



(b) total tokens per attempt (geometric mean over tasks, 95% bootstrap CI)

Figure 2: Success and cost by architecture, separately for the two models.

Table 7: H1 secondary measures (top; mean per-task difference, 95% CI, Holm-adjusted p within each measure) and H5 (bottom; HEXAGONAL−NONE Δ log tokens by stratum, six tasks each, exact sign-flip test, smallest attainable two-sided p = 0.031). Successful-attempt tokens use only the tasks where both arms had a success (19, 17 and 18 tasks).

Measure	Contrast	Estimate	95% CI	p_{Holm}
log tokens, successful attempts	LAYERED−NONE	−0.000	[−0.145, 0.158]	1.000
	HEXAGONAL−NONE	0.005	[−0.206, 0.212]	1.000
	HEXAGONAL−LAYERED	0.022	[−0.104, 0.138]	1.000
model calls	LAYERED−NONE	1.39	[−0.71, 3.49]	0.459
	HEXAGONAL−NONE	1.91	[−0.01, 4.04]	0.274
	HEXAGONAL−LAYERED	0.52	[−0.74, 1.84]	0.460
files modified	LAYERED−NONE	1.93	[1.05, 2.87]	<0.001
	HEXAGONAL−NONE	3.05	[1.95, 4.28]	<0.001
	HEXAGONAL−LAYERED	1.11	[0.01, 2.28]	0.072
wall time (s)	LAYERED−NONE	6.9	[−42.7, 61.0]	0.805
	HEXAGONAL−NONE	−59.1	[−106.1, −16.9]	0.028
	HEXAGONAL−LAYERED	−66.1	[−118.2, −22.7]	0.017
recomputed USD ($\times 10^{-3}$)	LAYERED−NONE	−0.02	[−3.01, 3.11]	0.990
	HEXAGONAL−NONE	−1.68	[−5.21, 2.40]	0.815
	HEXAGONAL−LAYERED	−1.66	[−4.11, 0.96]	0.669
H5: HEXAGONAL−NONE Δ log tokens by stratum			(column 5: unadjusted p)	
D domain rule		0.130	[−0.147, 0.417]	0.438
F feature		−0.053	[−0.188, 0.125]	0.563
C cross-cutting		−0.144	[−0.284, −0.019]	0.125
I integration		0.013	[−0.092, 0.123]	0.844
(F, C, I) − D		−0.192		0.117

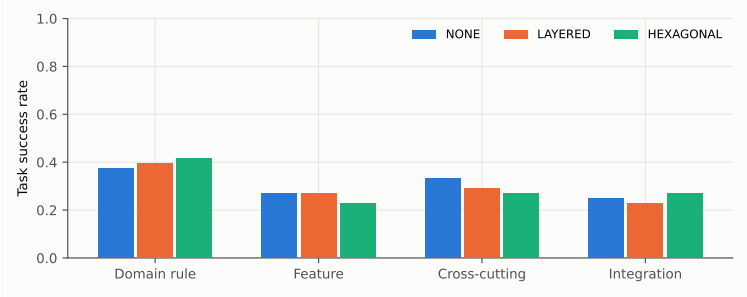


Figure 3: Task success rate by task stratum and architecture, pooled over both models (descriptive).

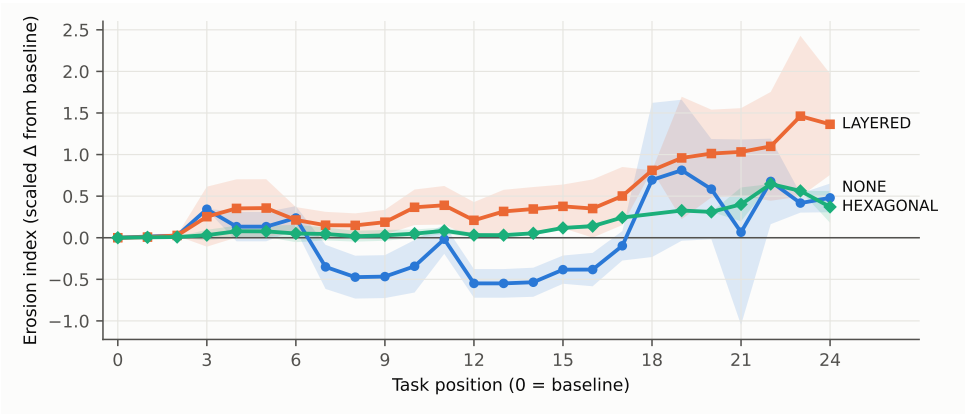


Figure 4: Erosion index $E_{t,k}$ over task position: mean and 95% band (normal approximation). Each point averages the trajectories (both models) whose snapshot compiles at that position. After position 3 almost all weak-model trajectories have dropped out (one NONE trajectory compiles occasionally), so later positions mostly reflect the strong model, and the set of contributing trajectories changes from point to point.

success to lose, so the interaction is bounded by the floor, not by the absence of an effect. The GLMM interaction terms (LAYERED×weak −1.68, HEXAGONAL×weak −0.83 log-odds, posterior SD 0.72 and 0.58) rest on the same 11 weak-model successes, and we do not interpret them. The weak model had passed the preregistered floor rule (≥30% success in NONE) on the three pilot tasks, which were single tasks on an unmodified baseline. Under the cumulative protocol it broke the build within the first three tasks of every trajectory and almost never repaired it (§4.1). We return to this in §5 and §7.

4.6 H2: Erosion

The index used all seven components (cycles, mean and max CBO, mean LCOM, mean distance from the main sequence, static warnings, duplicated lines); none was dropped for zero pooled spread. Table 8 compares slopes, and Figure 4 shows the index over task position. H2 is *not supported*. Neither structured variant eroded more slowly than NONE: the mean HEXAGONAL slope equals the NONE slope (0.0093 vs. 0.0091 index units per task, $p = 0.99$), and the LAYERED slope is about seven times larger (0.062), the opposite of the prediction, though not significantly so ($p_{\text{Holm}} = 0.34$). The one significant contrast is HEXAGONAL vs. LAYERED: the hexagonal code base eroded more slowly than the layered one (difference −0.052 per task, $p = 0.010$, $p_{\text{Holm}} = 0.031$). H2 did not predict a direction for this contrast, so we read it as a finding about the two architectures, not as support for H2.

The secondary LMM agrees in direction: the LAYERED×task-index interaction is 0.082 per task (95% CI $[0.009, 0.154]$, $p = 0.028$) and the HEXAGONAL×task-index interaction 0.016 ($p = 0.66$). The LMM optimizer did not converge cleanly (gradient warning, estimate on the boundary of the parameter space), so we treat this p-value as indicative only. Two features of the data qualify the slopes. First, the weak-model trajectories contribute slopes fitted on only 2–6 compiling snapshots, and one of them (LAYERED, weak, R4: slope 0.27 over five tasks) raises the pooled LAYERED mean. Restricted to the strong model (exploratory), the mean slopes are 0.013 (NONE), 0.050 (LAYERED) and 0.015 (HEXAGONAL); LAYERED exceeds both NONE and HEXAGONAL with an exact permutation $p = 0.057$ in each case (4 vs. 4 trajectories; smallest attainable $p = 0.029$). At the last compiling snapshot the strong-model index stood at 0.48, 1.11 and 0.37 on average. Second, all 12 strong-model trajectories ended above zero, so the code eroded in every arm; architecture changed the rate.

Table 9 breaks the strong-model slopes down by component (exploratory). The LAYERED slope is larger than the HEXAGONAL slope in all seven components and larger than the NONE slope in six, all but maximum CBO; the

Table 8: Trajectory-level tests: mean per-trajectory OLS slope per task for each arm of the contrast, and label-permutation tests stratified by model (8 vs. 8 trajectories). H2: erosion index. H6: own-architecture conformance violations; the last row is the one-sample sign-flip test of mean slope ≠ 0 over the 16 LAYERED and HEXAGONAL trajectories.

Contrast	slope (first arm)	slope (second arm)	p	p_{Holm}
H2 erosion index, directional				
LAYERED−NONE	0.0615	0.0091	0.170	0.341
HEXAGONAL−NONE	0.0093	0.0091	0.993	0.993
HEXAGONAL−LAYERED	0.0093	0.0615	0.010	0.031
H6 conformance violations				
HEXAGONAL−LAYERED (two-sided)	0.288	1.213	0.553	n/a
slope > 0, LAYERED and HEXAGONAL	0.751 [0.078, 1.810]		0.0625	n/a

Table 9: Exploratory: erosion components for the strong model (4 trajectories per arm). Left: mean change from baseline to the last compiling snapshot, in raw units. Right: mean standardized slope per task (×100), i.e. each component's contribution to the index slope (the index slope is the mean over the seven components).

	Δ baseline → last			standardized slope ×100		
Component	NONE	LAYERED	HEXAGONAL	NONE	LAYERED	HEXAGONAL
package cycles	0.00	0.25	0.00	0.0	4.5	0.0
mean CBO	0.18	0.26	0.05	−0.7	1.0	−0.3
max CBO	10.75	10.00	5.75	7.7	7.2	4.4
mean LCOM	−0.18	0.41	0.53	−4.1	4.3	2.7
mean distance D	−0.07	0.08	−0.04	−3.6	4.6	−2.5
static warnings	33.75	45.00	29.00	7.9	10.2	5.9
duplicated lines	17.25	39.75	0.00	1.7	3.3	0.0
index slope (×100)				1.3	5.0	1.5

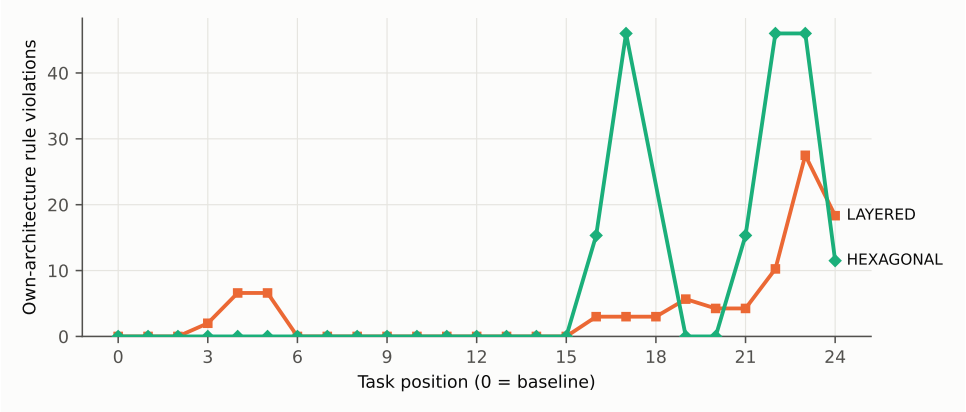


Figure 5: Own-architecture rule violations over task position (LAYERED, HEXAGONAL), averaged as in Figure 4. The HEXAGONAL peaks at positions 16–17 and 22–23 come from a single trajectory (strong model, R3) at positions where the other HEXAGONAL trajectories did not compile; the other three strong-model HEXAGONAL trajectories had zero violations throughout.

largest contributions come from static warnings, package cycles, distance from the main sequence, LCOM and duplicated lines. Cycles appeared only in LAYERED (in 6 strong-model and 3 weak-model snapshots). HEXAGONAL grew least in maximum coupling and gained no duplicated code at all: its CPD count stayed at zero in every compiling snapshot of every trajectory, while LAYERED gained 40 and NONE 17 duplicated lines on average.

4.7 H6: Conformance

Own-architecture violations rose by 1.21 per task in LAYERED and 0.29 in HEXAGONAL on average (difference −0.93, $p = 0.553$; Table 8, Figure 5). Both mean slopes are positive, but the one-sample test of a non-zero mean slope gives $p = 0.0625$ (separately: LAYERED $p = 0.125$, HEXAGONAL $p = 1.00$). H6 is therefore *not supported* at $\alpha = 0.05$: the trend is upward, and LAYERED and HEXAGONAL do not differ. The test had almost no room to succeed. Only five of the 16 trajectories ever had a violation in a compiling snapshot, and with five non-zero slopes $p = 0.0625$ is the smallest value the two-sided sign-flip test can return. The violations were concentrated. In LAYERED, three of four strong-model trajectories ended with violations (16, 39 and 6) and one weak-model trajectory reached 33 within five tasks. In HEXAGONAL, only one trajectory (strong, R3) ever violated a rule, ending

Table 10: Post-hoc validity checks (not preregistered; DEVIATIONS.md, 2026-10-15).

Check	Scope	Result
Reference solutions	T01–T24, built cumulatively in each of the three variants, graded by the experiment's grader	24/24 tasks pass in all three variants, 0 regressions; no hidden test is faulty
Baseline validation	24 tasks × 3 baselines	72/72 OK (FAIL_TO_PASS fail, guards and baseline suite pass) after re-running two OOM-killed checks
Re-grading	all 403 recorded failures, re-graded at low load	0 outcomes changed; built, started and task tests passed identical in every case
Quality review	38 successful attempts (27 strong, 11 weak), stratified random sample	strong: 26 genuine, 1 partial, 0 gaming; weak: 5 / 4 / 2; architecture fits in 21 of 24 LAYERED/HEXAGONAL attempts

at 46. The rules violated most often were, in LAYERED, `fhirModelOnlyInWeb` (HAPI FHIR types outside the web layer; 17 snapshots, 3 trajectories) and `layersAreRespected` (9 snapshots, 2 trajectories), and in HEXAGONAL `applicationDependsOnlyOnDomainAndItself` and `fhirModelOnlyInWebAdapter` (6 snapshots each, both in the one trajectory).

4.8 Exploratory analyses

The registration lists regressions, localization and qualitative coding as exploratory (PREREGISTRATION §6.6). We add some descriptive breakdowns by model, because the weak arm's floor makes pooled numbers hard to read. None of these were planned as confirmatory tests.

Build failures. For the strong model, the graded snapshot failed to build in 11.5% of NONE, 15.6% of LAYERED and 30.2% of HEXAGONAL attempts (HEXAGONAL−NONE +18.8 pp per task, 95% CI [9.4, 29.2], sign-flip $p = 0.001$, uncorrected). The grader also compiles the agent's test sources. Counting only main code that does not compile (the erosion measurement), the rates are 10.4%, 10.4% and 22.9%. The HEXAGONAL failures cluster late: every strong-model HEXAGONAL trajectory failed to build at T17, T18 and T19. In one of them (R1) the build stayed broken from T17 to T24, which ended the run of successes in that trajectory.

Regressions. Attempts that broke at least one previously passing test were rare: 2, 6 and 5 of 96 strong-model attempts in NONE, LAYERED and HEXAGONAL (4, 0 and 0 for the weak model, whose snapshots rarely built). The number of regressed tests was 21, 161 and 54 in the strong model, but it is dominated by single attempts: two consecutive LAYERED attempts (R1, T19 and T20) account for 122 of the 161.

Change footprint. The strong model modified 4.8, 6.7 and 9.5 files per attempt on average in NONE, LAYERED and HEXAGONAL, i.e. about twice as many in HEXAGONAL as in NONE, at the same token cost and success rate.

Conformance by trajectory. Three of four strong-model HEXAGONAL trajectories never violated any of their own rules in any compiling snapshot. Three of four strong-model LAYERED trajectories ended with violations.

Use of version control by the agent. About half of the transcripts (279 of 576) contain `git stash`, `git checkout` or `git reset`. Agents used `git stash` to compare test results with and without their change. One agent removed a graded commit from the history (deviation 4).

Localization and command mix. We classified every bash command in the transcripts by keyword (read, search, edit, build or test, git, other) and, for each attempt, listed the Java files the agent read or searched before its first edit (`analysis/localization.py`; descriptive only). For the strong model, reading and searching made up 62% of the commands, editing 14% and builds or test runs 11%. Before its first edit the agent ran 24.1, 24.4 and 32.8 commands and touched 11.4, 19.0 and 28.1 distinct Java files in NONE, LAYERED and HEXAGONAL. Localization recall, the share of the main-code files it later changed that it had read before its first edit, was similar: 0.80, 0.76 and 0.76. The hexagonal agent thus read about two and a half times as many files to find the same share of its edit targets, and it ran builds or tests less often (7% of its commands vs. 12% in NONE and 14% in LAYERED), which is consistent with, but does not establish, a link to its higher rate of non-building snapshots. The weak model read less (5–8 files before its first edit) and reached a recall of about 0.59 in every variant. The keyword classification is coarse: a command that both searches and edits counts once, by the first matching rule. *Qualitative coding* of failed attempts is deferred; the released transcripts make it possible for others to do it.

4.9 Post-hoc validity checks

After data collection we ran four checks on the measurement itself (Table 10). None was preregistered. All are logged in DEVIATIONS.md (2026-10-15), and none changes a confirmatory result. The preregistration had planned reference solutions for every task. At registration only a partial one existed (12 tasks, NONE), and the complete set was built only after data collection. Two Phase 1 results prompted the checks: no agent attempt solved T18 or T23, and, while the reference solutions were being built, the grader produced false failures under heavy Docker load.

Reference solutions. Three AI agents, one per variant, implemented T01–T24 in task order, each task on top of the previous one, as the experimental agents do. Every commit was graded with the experiment's grader against the hidden tests of all tasks so far (`harness/grade_one.py`; `data/reference/`). All 24 tasks passed in all three variants with no regressions. No hidden test demands something its ticket or the specification does not say, and no two tests contradict each other. T18 (paging) is solvable with a straightforward implementation that passed on its first grading. The pitfalls lie in details the ticket states: the server must accept its own paging parameter in the `next` link while still rejecting unknown parameters, and must emit no `next` link on an exactly full last page. T23 (AuditEvent) contains a library trap. HAPI's JSON encoder strips `/_history/<version>` from references by default, so an implementation that builds the versioned reference the ticket requires (`Patient/12/_history/2`) still serializes `Patient/12`, and 5 of the 7 T23 tests fail. The fix is one parser option (`setDontStripVersionsFromReferencesAtPaths`). The test is correct, but the cause is visible only in the serialized JSON. One ticket has a wording flaw: T21 lists `corrected` as an allowed successor of `final`, but `corrected` lies outside the supported status subset and is rejected with 400 by clarification C11. The hidden test does not exercise it.

Baseline validation. We ran the task validation for all 24 tasks on all three baselines (`data/validate_all24.log`). All 72 checks passed. Two checks (HEXAGONAL, T22 and T24) first failed because the application's JVM in the grading container was killed for lack of memory while three reference builders shared the Docker VM. Re-run at low load, both passed.

Re-grading. The false failures under load looked like this: the application's JVM died silently during the run, and every test then failed with a refused connection, or dozens of unrelated tests failed and passed again on an identical re-run. Phase 1 graded with two to three workers in parallel, so the same could have happened there. We therefore re-graded all 403 attempts recorded as failures, with the same grader at low load (`harness/regrade.py`, `data/phase1/regrade.jsonl`). No outcome changed. Whether the snapshot built, whether the application started and how many task tests passed were identical in every case. All 10 recorded "application did not start" cases show real exceptions in agent code. Load can only produce false failures, not false passes, so the 173 successes were not re-graded. Phase 1 grading was not affected by the flakiness, and no sensitivity analysis is needed.

Quality review. A pass on the hidden tests does not show that a ticket was implemented in general rather than for the tested inputs. We drew a stratified sample of successful attempts (`data/review/`, seed 20261015): 10 at random per variant, plus every remaining weak-model success, $N = 38$ (27 strong, 11 weak). One reviewer, an AI agent, read each diff against the ticket and a fixed rubric. *Genuine* implements the ticket generally, *partial* passes the tests but drops a requirement for realistic inputs, and *gaming* special-cases test data, hardcodes values, or weakens checks, tests or the build. For LAYERED and HEXAGONAL, a *major* architecture violation breaks an explicit rule of ARCHITECTURES.md, and a *minor* one puts code in the wrong package without breaking a dependency rule. Of the 27 strong-model successes, 26 are genuine and one is partial (LAYERED, R3, T10 silently drops addresses without a city); none games the tests. The weak model's 11 successes are weaker: 5 genuine, 4 partial, 2 gaming. Four of its five T01 successes check `Quantity.unit` instead of the UCUM `code` the ticket names. The hidden T01 test sets both fields to the same value and cannot tell them apart, a coverage gap rather than a wrong test. The two gaming cases are T02 attempts in NONE trajectories that a failed T01 had left uncompilable. Both restored the build by disabling baseline validation (commenting out `validate(observation)`, deleting `requireSupportedCodes` calls), and both caused regressions. Nine of the 11 weak-model successes left collateral such as backup files or placeholder tests. The architecture fits in 21 of the 24 sampled LAYERED and HEXAGONAL attempts. The one major violation (LAYERED, strong, R4, T16: HAPI `FhirContext` and `Parameters` imported into `service`) is also recorded by the conformance measure (six new `fhirModelOnlyInWeb` violations at that snapshot). The two minor ones are both HEXAGONAL T04 attempts that put the outbound notifier in `adapter.in.web` instead of behind an outbound port. Five of the 38 sampled successes (12 of all 173) also caused regressions, which are recorded separately as preregistered.

5 Discussion

Summary. Of the five interpretable preregistered hypotheses, none was supported. Architecture did not change success (H3) or token cost (H1, H5) in a way this design could detect. The structured variants did not erode more slowly than the unstructured one (H2), and conformance violations did not significantly accumulate (H6). The capability interaction (H4) is not interpretable. Two results nevertheless stand out. The hexagonal code base eroded significantly more slowly than the layered one, and the layered one eroded fastest of all three. And the agent touched about twice as many files in HEXAGONAL without paying for it in tokens or success.

No success or token penalty for a capable agent. One might expect indirection to tax an agent that reads code through a shell: more files and more hops between a request and the logic it reaches. For a capable model working cumulatively on a Java service we found no such penalty. The structured variants cost the agent more edits (Table 7), but not more tokens, calls or failures. Several factors could explain this: model capability, framework conventions (Spring's layered and hexagonal idioms are common in training data), and naming (our variants follow a well-known reference style with descriptive names). Our design cannot separate them. Trivedi and Schmitt [36] likewise find that a code property can leave pass rates unchanged while shifting effort, and Stoyanov's single-system comparison [32] reported a slower hexagonal variant, whereas our secondary wall-time measure points the other way. Two cautions apply. First, a null result here is a limited one. For the strong model, the CI for HEXAGONAL−NONE success runs from −12.5 to +10.4 pp, and most attempts used the full 60-call budget, so the token measure is censored: within a fixed budget, tokens mostly reflect context size per call, and a variant that needs more calls cannot show it. Second, the study has one domain and one capable model.

What the success measure captures. The post-hoc checks (§4.9) support reading success as agent capability. Every task is solvable in every variant, so the 0% success on T18 and T23 is a failure of the agents, not of the tests. Most strong-model T18 attempts (10 of 12) left a snapshot that did not build. T23 adds a library trap: the HAPI encoder drops the version from the audited reference, and this shows only in the serialized JSON, which an agent sees only if it inspects the actual response rather than the object it built. In Phase 1 no agent got that far; the three strong-model T23 snapshots that built and started passed none of the seven tests, including those the trap does not affect. Successes are real too: 26 of 27 sampled strong-model successes are genuine implementations, and none games the tests. The success rates in Table 5 therefore measure working code, not test fitting, at least for the model that carries the results. One hidden test is weaker than its ticket. T01 cannot distinguish a check on the UCUM `code` from one on the display `unit`, and four of the weak model's 11 successes exploit exactly this gap, which slightly overstates the weak model's already minimal success.

Architecture changed the fate of the code. Erosion grew in every arm: all 12 strong-model trajectories ended with a higher index than they started with, and static warnings rose in every one of them. This is consistent with the notion of agentic entropy [5]: agents that solve tickets one at a time leave a structurally worse code base behind. Architecture modulated the rate. The layered variant eroded fastest on almost every component, including duplication, warnings and the only package cycles in the study. Its conformance broke in three of four strong-model trajectories, mostly by HAPI FHIR types leaking out of the web layer. The hexagonal variant held its structure: no duplicated code was added, three of four strong-model trajectories never violated a hexagon rule, and its index slope matched that of NONE. The quality review offers convergent evidence for the conformance measure: reading the code independently, the reviewer found the architecture intact in 21 of 24 sampled LAYERED and HEXAGONAL successes, and the one major violation it found was also counted by the ArchUnit rules. The two minor misplacements it found break no dependency rule, so the conformance count, by design, misses them.

We can offer only hypotheses for this difference. In HEXAGONAL the boundaries are visible in the code the agent reads. Ports are interfaces, adapters are named as such, and the domain has its own model without HAPI types, so an agent that imitates nearby code follows the boundary. In LAYERED the boundaries are soft: controller, service and repository are all Spring beans, HAPI types are available everywhere on the classpath, and nothing local signals that a service should not return a FHIR resource. The agent can take the shortcut without noticing that it crosses a layer. This reading fits *Architecture as Capability Equalizer* [4], which finds that explicit architectural specifications help agents. A hexagonal code base may act as an implicit specification. Testing this needs the localization and qualitative analyses, which are still to be done.

There is a price, and alternative explanations. HEXAGONAL attempts by the strong model more often left a project that did not build (30% vs. 11% in NONE; exploratory). A change in HEXAGONAL has to be threaded consistently through more places (domain model, port, service, mapper, adapter), which gives more chances to stop halfway when the call budget runs out. Non-compiling snapshots are left out of the slope fits, so if the snapshots that fail to build in HEXAGONAL are also the most eroded ones, the HEXAGONAL slope is flattered. Some components are means over classes, and HEXAGONAL has about four times as many classes as NONE, so a local change moves its means less. The duplication, warning and conformance counts are not means and show the same pattern, which makes a purely mechanical explanation less likely, but it remains a confound of the size difference that is part of the treatment (§6).

The weak model and the floor. The weak model failed at the floor despite a preregistered floor rule designed to prevent it. The rule was checked on three pilot tasks, each a single change to a fresh baseline, where Qwen3-Coder-30B passed. Under the cumulative protocol the model broke the build within the first three tasks of every trajectory and almost never recovered, so most of its 288 attempts started from a broken project. The quality review shows

how it "recovered" when it did: both of its gaming successes restored a build broken by a failed T01 by disabling baseline validation, and both caused regressions. A model that repairs a build by switching checks off rather than fixing the cause is consistent with the collapse to the floor that we observed. The lesson is that a capability check must use the real protocol, here a short sequence of cumulative tasks, and that the capability question needs a mid-capability model whose success is clearly above the floor in the cumulative setting. The preregistered fallback, Qwen3-Coder-Next, is a candidate.

Next steps. Phase 2 repeats the design in a second domain (order management), where FHIR's external data model is absent. It will test domain generality (H7). It can also state the HEXAGONAL vs. LAYERED erosion difference, found here without a directional prediction, as a confirmatory directional hypothesis. It should replace the weak model with a mid-capability one. Study 2 (planned in `STUDY2-GUARDRAILS.md`) asks whether Copilot-style guardrails (architecture instructions, ArchUnit and static checks run as a hook before submit, self-review) control agentic entropy. Our results suggest two concrete questions for it: whether enforcement closes the gap between soft layered boundaries and explicit hexagonal ones, and whether build-and-check feedback reduces the non-compiling attempts that HEXAGONAL produced.

6 Threats to Validity

We follow the usual classification [39]. The first items were stated in the preregistration (§7.1). The rest were identified while running and analyzing the study.

Construct validity. *Size is part of the treatment.* HEXAGONAL has 1.81× the LOC and 3.9× the files of NONE. We report the profile and do not equalize it, so effects of architecture cannot be separated from the code volume the style implies. This also affects the erosion index: mean CBO, mean LCOM and mean distance from the main sequence are averaged over more classes or packages in HEXAGONAL, so the same local change moves them less. *Erosion index.* The index aggregates established metrics with equal weight. Some components, such as distance from the main sequence, depend on package structure and may behave differently by style. We therefore report the components separately (Table 9). Averaged components can also fall below their baseline simply because an agent adds many small classes: in strong-model NONE snapshots with a negative index, mean LCOM stood 2.7 scale units below baseline while maximum CBO stood 0.9 units above it. A negative index therefore does not mean the code became cleaner, and the averaged components favor styles that add many small classes, which is one more reason to read the component table next to the index. *Conformance* exists only for LAYERED and HEXAGONAL, and its rules differ between them, so violation counts are not on a common scale. *Cost* includes cached prompt tokens, which are cheap in money but still measure how much context the agent processes. We report recomputed USD as a secondary measure. *Step-limit censoring.* Cost is right-censored at 60 calls, and the censoring is heavy: 55.7%, 60.4% and 63.5% of attempts in NONE, LAYERED and HEXAGONAL hit the limit (strong model 57.3%, 64.6%, 65.6%; weak model 54.2%, 56.2%, 61.5%). Censoring does not bias the permutation test under the null, but it lowers the power of H1 and compresses differences between arms, and the limit itself shapes success. The null result for H1 means "no difference in tokens within a 60-call budget", not "no difference in the work needed". The secondary analysis on successful attempts is only partly a remedy, because success is itself an outcome. *Excluded snapshots.* Non-compiling snapshots are left out of the slope fits. They are frequent (96, 96 and 111 of 192 per arm, mostly from the weak model) and, for the strong model, twice as frequent in HEXAGONAL (22 vs. 10 and 10 of 96). The slopes therefore describe different subsets, and the weak-model slopes rest on 2–6 points each. *Success as passing hidden tests.* Tests can be passed without implementing a ticket. The quality review found no gaming among sampled strong-model successes, but it has limits: one reviewer, itself an AI agent, read the code without re-running it, knew each variant from the code structure, and the weak stratum is small (11 attempts, five of them T01).

Internal validity. *Construction bias.* One author, assisted by AI agents, built all variants and tasks, and no independent human review took place. Mitigations: a public reference style, the mechanical equivalence gate (79 black-box tests, identical schema and dependencies), frozen ArchUnit rules, published size profiles and full release of all materials. AI-built variants may also be closer to what models find natural than human-written code would be. *Task authoring.* Tickets are behavior-only and graded by identical black-box tests. The post-hoc reference solutions show that every hidden test can be satisfied in every variant and that no test contradicts its ticket or another test (§4.9). Residual limits: the reference solutions were written by AI agents, not by independent human developers, and no human outside the project reviewed the tickets. One ticket (T21) names a status the server cannot accept, and one hidden test (T01) is weaker than its ticket. *Grading reliability.* The grader produced false failures when the Docker VM ran out of memory. Re-grading all 403 recorded failures at low load changed no outcome, so this threat is resolved for Phase 1. *Provider drift.* Hosted models can change silently, and providers can re-route requests. We

pinned model IDs, interleaved all arms in every worker, and logged the provider per call: a single provider served each model throughout. Drift during the 34-hour window would affect all arms equally but could still interact with architecture. *Host interruption.* Collection stopped for about 5.4 hours, and the remaining R4 attempts ran in per-model workers. Each worker still interleaved the three architectures, so architecture comparisons within a model are unaffected; only the interleaving across models was lost for those attempts. The interrupted attempt was rolled back and rerun from its last commit. *Dependencies between tasks.* Tasks were designed to be independent, so the preregistered `dependency_failed` covariate is zero throughout and the sensitivity analysis is uninformative. The real dependency in a cumulative design is the shared code base: a broken build blocks all later tasks in its trajectory. This dominated the weak model and also affected the strong model (one HEXAGONAL trajectory stayed unbuildable from T17 to T24). Success rates of later tasks thus partly measure the ability to repair earlier damage. *Agent access to version control.* Agents could run git in the workspace, and one rewrote the branch history. Grading and (after deviation 4) erosion use the commit objects recorded by the harness, so no measurement depends on the agent leaving history intact.

External validity. *Single domain in Phase 1.* Results come from one application. FHIR is a public standard that models have probably seen often, and its external model makes the placement of mapping code especially salient; the LAYERED violations we observed were mostly about exactly that. Phase 2 (order management) tests generality (H7). We claim nothing about other languages, frameworks or much larger systems. *Effectively one model.* With the weak model at the floor, every substantive finding rests on one open-weights model, run without extended thinking in one scaffold. Frontier proprietary agents with richer tools may behave differently, and nothing can be said about how effects change with capability. *Scaffold.* A bash-only loop with a 60-call budget is deliberately minimal. Products with planning, retrieval, linting or self-review steps may close or widen any gap.

Conclusion validity. *Reduced sample.* The cost rule dropped R5 as preregistered, so each arm has 8 trajectories and each task 8 attempts per arm instead of 10. The registered simulation gives 4-run power of 0.88 (0.77 after Holm) for a 15 pp success difference, against 0.93 (0.86) with five runs (§3.6). *Realized sensitivity is lower still.* The simulation assumed a mean success near 75% with two informative models. The realized mean was 30%, and the weak arm contributed almost no information, so the effective sample for H3 is about half the nominal one. The CIs in Table 6 are the best guide to what is excluded. *Trajectory-level tests* have 8 vs. 8 trajectories, and the H6 one-sample test could not go below $p = 0.0625$ with these data (§4.7). *Mixed models.* Both LMMs reported convergence problems, so their p-values are indicative only. *Multiplicity.* Holm correction is applied within each hypothesis, not across hypotheses or across the secondary and exploratory analyses; the latter are descriptive and need confirmation. H5 per-stratum tests have six tasks each and are descriptive. *Contamination* is limited by keeping all materials private until data collection ended and by running without network access. The materials will become public afterwards.

7 Lessons for Running Agent Experiments on a Shoestring

The study ran on one laptop with Docker and a budget of USD 30. Most of the engineering effort went into problems that a well-funded lab may never see but that can silently bias a low-budget experiment. We record them because each one would have turned infrastructure behavior into apparent agent behavior. Lessons 1–8 were learned before registration, 9–11 during Phase 1, and 12–13 in the post-hoc checks.

1. Free endpoints are not infrastructure. We first planned to use free (`:free`) endpoints. `qwen/qwen3-coder:free` was withdrawn on 2026-10-11, and `poolside/laguna-s-2.1:free` was throttled for many minutes per call. Both were dropped before any confirmatory data in favor of the paid endpoints of the same models. With provider-side prompt caching the paid runs were cheap: 83% of prompt tokens in the pilot and 96–97% in the shakedown and 91.7% in Phase 1 were served from cache, and Phase 1 cost USD 24.25 in total, about USD 0.04 per attempt. Open weights mean the study stays reproducible when endpoints change.

2. Providers may ignore reasoning settings. With extended thinking enabled, the provider ignored both `reasoning.max_tokens` and `effort: low`. Calls ran to the output cap without a tool call, and the scaffold counted them as format errors. Six of 17 shakedown attempts ended this way, a failure mode unrelated to the code base that would have added noise to every arm. Turning thinking off removed it (0 format errors in 120 calls). Measure the format-error rate in a shakedown, and fix reasoning settings before registration on reliability grounds alone.

3. Errors can arrive with HTTP 200. Some upstream errors came back as HTTP 200 with an `error` body and no `choices`. The stock scaffold crashed on them with a `KeyError`. A thin wrapper turns them into retryable API

errors and raises the request timeout from 60 to 300 s. More generally, the harness keeps an explicit allow-list of exit states that count as agent outcomes. Everything else is infrastructure: it is rolled back to the last commit and retried, never scored.

4. Command timeouts do not kill processes. When a command times out, the scaffold stops waiting, but a hung `mvn test` or a server the agent started in the background can keep running inside the container. It then competes for the 2-CPU budget and inflates the wall time of later commands. Wrapping every command in `timeout -k 5 590` inside the container, 10 s before the scaffold's own 600 s limit, and removing the container after each attempt fixed this.

5. Define "regression" with care. In a cumulative design, the naive definition (any earlier hidden test that fails now) counts every earlier task the agent never solved as a regression in every later attempt. We define a regression as a test that passed at the previous grading and fails now, record "still failing" tests separately, and compare against the last snapshot that built and started.

6. Cumulative grading needs *supersedes*. Legitimate tickets change baseline behavior: a new search parameter that used to be rejected, a stricter rule that an old test violates, an extended CapabilityStatement that an old test checks exactly. Without an explicit list of superseded tests, correct solutions would register as regressions. Nine of 24 tasks needed such entries. A full shakedown trajectory over all 24 tasks, run before registration, found these conflicts. Configuration must also accumulate, so that earlier integration tasks keep being tested against their mock servers.

7. Make every step resumable and idempotent. Long runs on a laptop get interrupted. If the process is killed between writing a result row and saving trajectory state, a naive restart grades the same task twice. The runner reconciles state with the results file at startup. Measuring erosion post hoc from git snapshots, rather than in the loop, means a crashed or upgraded measurement tool never costs agent runs.

8. Pin and adapt the measurement tools. CK 0.7.0 predates Java records. It either mis-parsed Java 21 syntax or lost every type in a file that contains a record (22 files in the HEXAGONAL baseline, 3–4 in the others). Without the desugaring step (§3.5), coupling and cohesion would have been computed on an incomplete set of types that differs by variant. We pinned tool versions in a separate image tag, checked release digests, and verified that CK reports no errors on the baselines.

9. Test the capability floor under the real protocol. Our floor rule checked the weak model on three single tasks against a fresh baseline, and it passed. In the cumulative protocol the same model broke the build within three tasks in all 12 trajectories and almost never repaired it, which made H4 uninterpretable. A floor check should run a short cumulative sequence (several tasks, each on top of the last) and require a clear margin above the floor, so that a mid-capability model is chosen.

10. The agent shares your version control. About half of the transcripts contain `git stash`, `checkout` or `reset`, and one agent ran `git reset --hard` and removed the previous task's commit from the branch. The harness must record the commit object it grades and measure exactly that object later, rather than walk the history the agent can rewrite. The sandbox isolates the network, not the repository: inside it the agent owns `.git`.

11. Make outcome-blind decisions final. The cost rule fired once, as designed, and the host stopped hours later. On restart, a runner that re-evaluated the rule from scratch could have reached a different decision on more data. The orchestrator now records the decision and treats it as final across restarts, and the restart itself is logged as a deviation.

12. Memory overcommit makes graders fail silently. On a laptop, Docker runs all containers in one VM with a fixed memory size (5.3 GiB here). Several grading containers of 3 GB each fit on paper but not in practice. When the VM runs out of memory, the kernel kills the application's JVM inside a grading container without any error in the grader's output, and every following test fails with a refused connection. The result looks like an agent that broke everything. Our Phase 1 grading was verified clean by re-grading all 403 failures (§4.9), but we found the problem only by chance, while building reference solutions in parallel. Future runs should cap parallel grading by the memory available in the VM, not by CPU count, and should treat a verdict in which every test fails with a connection error as an infrastructure failure to be retried.

13. Build the reference solutions before registration. We planned them but completed them only after data collection. They then showed that all tasks are solvable, but they also surfaced the T23 library trap and the T21 wording flaw. Before registration, both could have been fixed or deliberately kept and documented. Reference builds also exercise the grader at scale, which is how lesson 12 was found.

8 Conclusion

We set out to test, under preregistered control, whether layered and hexagonal architecture help or hinder AI coding agents that evolve a Java service over many tasks, and whether the agents preserve the architecture. For a capable open-weights agent working through 24 cumulative tickets on one FHIR service, architecture changed neither task success nor token cost, although the hexagonal variant required changes to about twice as many files. None of the preregistered directional predictions was supported: the structured variants did not erode more slowly than the unstructured one, and conformance violations did not significantly accumulate. Architecture did shape what happened to the code: the layered code base eroded fastest, and the hexagonal one eroded significantly more slowly than the layered one and mostly kept its own rules, at the price of more attempts that left the project unbuildable (exploratory). These results come from one domain, one capable model and four runs, and the weaker model was at the floor. Phase 2 tests the same design in an order-management domain with a mid-capability second model, and Study 2 tests whether Copilot-style guardrails (architecture instructions, enforced ArchUnit checks, self-review) slow the erosion we observed.

Data availability. All materials are released under MIT (code) and CC-BY 4.0 (text, data): the three variants, specification and clarifications, tasks, hidden tests, harness, agent and model configuration, sandbox Dockerfiles, raw agent transcripts, per-task git snapshots, results and erosion tables, and analysis scripts. All data are synthetic. OSF project: `https://osf.io/rneym`. Registrations: `https://osf.io/r5t3d` (Phase 1) and `https://osf.io/w467e` (idea). Code and materials: `https://github.com/Pisarews/architecture-coding-agents`; the frozen protocol is the git tag `prereg-v1`. Transcripts, snapshots and result tables are stored in the OSF project.

Use of AI tools. AI coding agents built the three variants from the author's specification under the author's review (§3.1). The author used an AI assistant (Claude, Anthropic) to write the harness, metrics pipeline and analysis scripts, to run the experiment, to carry out the AI quality review of successful attempts (§4.9), and to draft and edit this manuscript. The author designed the study, made the methodological decisions, reviewed the code, checked the reported numbers against the data, and takes responsibility for the content.

References

[1] Maurício Aniche. Java code metrics calculator (CK). `https://github.com/mauricioaniche/ck`, 2015. Version 0.7.0 used.

[2] Markus Borg, Nadim Hagatulah, Adam Tornhill, and Emma Söderberg. Code for machines, not just humans: Quantifying AI-friendliness with code health metrics. In *Proceedings of the 3rd ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*, 2026. arXiv:2601.02200.

[3] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal. *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Wiley, 1996.

[4] Arquimedes Canedo. Architecture as capability equalizer for coding agents. arXiv:2608.21747, 2026.

[5] Matteo Casserini, Alessandro Facchini, and Andrea Ferrario. Beyond the 'diff': Addressing agentic entropy in agentic software development. arXiv:2604.16323, 2026. Position paper, Human-Centered Explainable AI (HCXAI) Workshop at CHI 2026.

[6] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.

[7] Alistair Cockburn. Hexagonal architecture. `https://alistair.cockburn.us/hexagonal-architecture/`, 2005.

[8] Lakshitha de Silva and Dharini Balasubramaniam. Controlling software architecture erosion: A survey. *Journal of Systems and Software*, 85(1):132–151, 2012.

[9] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[10] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[11] Phillip Good. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer, 3rd edition, 2005.

[12] HL7 International. FHIR release 4 (v4.0.1). `https://hl7.org/fhir/R4/`, 2019.

[13] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[14] Tom Hombergs. *Get Your Hands Dirty on Clean Architecture*. Packt, 2nd edition, 2023. Reference implementation: `https://github.com/thombergs/buckpal`.

[15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[16] Tue Le, Minh V. T. Thai, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470. https://github.com/SWE-EVO/SWE-EVO, 2025.

[17] Meir M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980.

[18] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. Understanding architecture erosion: The practitioners' perceptive. In *Proceedings of the IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, pages 311–322, 2021. doi: 10.1109/ICPC52881.2021.00037.

[19] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. Understanding software architecture erosion: A systematic mapping study. *Journal of Software: Evolution and Process*, 34(3):e2423, 2022.

[20] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2002.

[21] Bardia Mohammadi, Lars Klein, Aman Chadha, Akhil Arora, and Laurent Bindschaedler. The working set of a coding agent: Coherence debt in repository-scale tasks. arXiv:2608.16630, 2026.

[22] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.

[23] Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. arXiv:2603.24755. https://github.com/SprocketLab/slop-code-bench, 2026.

[24] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12): 1053–1058, 1972.

[25] Dewayne E. Perry and Alexander L. Wolf. Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52, 1992.

[26] PMD contributors. PMD: An extensible cross-language static code analyzer. https://pmd.github.io, 2026. Version 7.28.0 used, including CPD.

[27] Poolside. Laguna S 2.1 model card. https://huggingface.co/poolside/Laguna-S-2.1; served via OpenRouter as `poolside/laguna-s-2.1`, 2026. 118B total / 8B active parameters, OpenMDW-1.1 license.

[28] Qwen Team. Qwen3 technical report. arXiv:2505.09388, 2025.

[29] Grigory Sapunov. Theory of code space: Do code agents understand software architecture? arXiv:2603.00601, 2026.

[30] Tyler Slater. Quantitative analysis of technical debt and pattern violation in large language model architectures. arXiv:2512.04273, 2025.

[31] SpotBugs contributors. SpotBugs: Find bugs in Java programs. https://spotbugs.github.io, 2026. Version 4.10.4 used.

[32] Kristiyan Stoyanov. I expected hexagonal architecture to help my coding agent. It didn't. Blog post, https://dev.to/kstoyanovai/i-expected-hexagonal-architecture-to-help-my-coding-agent-it-didnt-3blg, 2026. Grey literature. Technical report: *Hexagonal architecture and coding agents: an exploratory study of change acceptance*, Zenodo, https://doi.org/10.5281/zenodo.22775539.

[33] SWE-agent team. mini-swe-agent: The 100 line AI agent that solves GitHub issues or helps you in your command line. https://github.com/SWE-agent/mini-swe-agent, 2025. Version 2.4.6 used.

[34] SWE-bench team. SWE-bench bash only leaderboard. https://www.swebench.com, 2025. Runs in the mini-SWE-agent environment. Accessed 2026-10.

[35] TNG Technology Consulting. ArchUnit: Unit test your Java architecture. https://www.archunit.org, 2025. Version 1.5.1 used.

[36] Priyansh Trivedi and Olivier Schmitt. Does code cleanliness affect coding agents? A controlled minimal-pair study. arXiv:2605.20049, 2026.

[37] Dmytro Ustynov. Beyond human-readable: Rethinking software engineering conventions for the agentic development era. arXiv:2604.07502, 2026.

[38] Jason Walonoski, Mark Kramer, Joseph Nichols, Andre Quina, Chris Moesel, Dylan Hall, Carlton Duffett, Kudakwashe Dube, Thomas Gallagher, and Scott McLachlan. Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association*, 25(3):230–238, 2018.

[39] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.

[40] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

[41] Daoguang Zan, Zhirong Huang, Ailun Yu, Shaoxin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu,

Dezhi Ran, Muhan Zeng, Bo Shen, Pan Bian, Guangtai Liang, Bei Guan, Pengjie Huang, Tao Xie, Yongji Wang, and Qianxiang Wang. SWE-bench-java: A GitHub issue resolving benchmark for Java. arXiv:2408.14354, 2024.

[42] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, et al. Multi-SWE-bench: A multilingual benchmark for issue resolving. arXiv:2504.02605, 2025.